

## PM Accelerator Mission

By making industry-leading tools and education available to individuals from all backgrounds, we level the playing field for future PM leaders. This is the PM Accelerator motto, as we grant aspiring and experienced PMs what they need most – Access. We introduce you to industry leaders, surround you with the right PM ecosystem, and discover the new world of AI product management skills.

## Weather Trend Forecasting Report

### Introduction

**Weather Trend Forecasting** focuses on predicting weather conditions using historical data, including features such as temperature, air quality, sunrise, and sunset. Before training the models, I transformed the raw weather dataset into a modeling dataset to ensure the models could effectively learn patterns and make accurate predictions.

To forecast weather conditions, I implemented three algorithms: **Random Forest**, **XGBoost**, and a **stacking method**. Among these, the **Random Forest** algorithm achieved the best performance, with an accuracy of **57%**.

### Methodologies

#### Data Cleaning

##### Missing values

No missing values are found, I don't need to think how I could handle missing values.

##### Outliers

I used the winsorization method to deal with outliers. I replaced bottom 5% with its nearest bottom 5% value and replaced top 5% with its nearest top 5% value.

Note: Handling outliers is a difficult task to do in weather forecasting because we know that extreme factors lead to extreme weather conditions. If we remove or replace them with only mean or median, predictive models lose the ability of predicting extreme weather conditions.

Moreover, handling outliers in this dataset should be location-specific. For example, wind speeds can vary significantly by region, so an outlier like 100 mph in Kabul should be

corrected by referencing the wind data from the last 10 days in the same location. When handling outliers, it's important to consider only the data corresponding to the specific location, ensuring that the correction is contextually appropriate for that region.

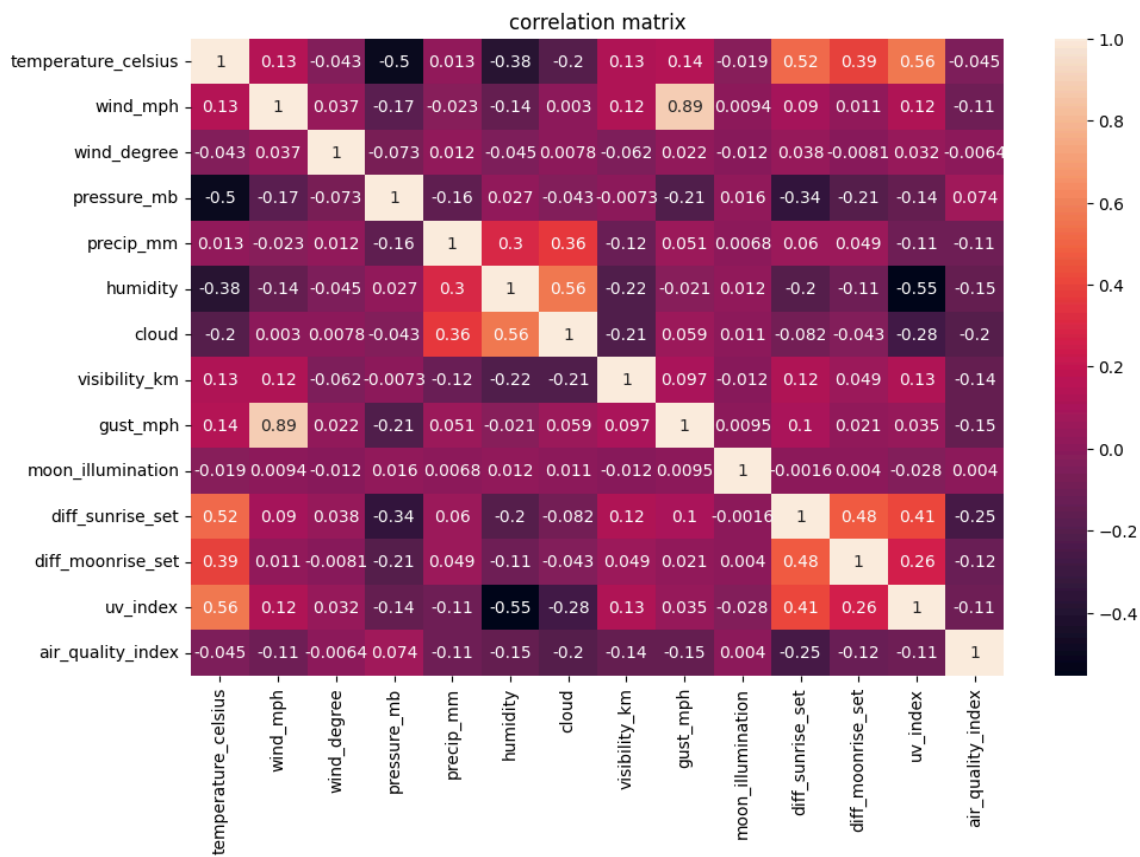
### **Normalization**

I imported `StandardScaler()` from `scikit-learn` to standardize the numerical features which has 0 variance and 1 mean.

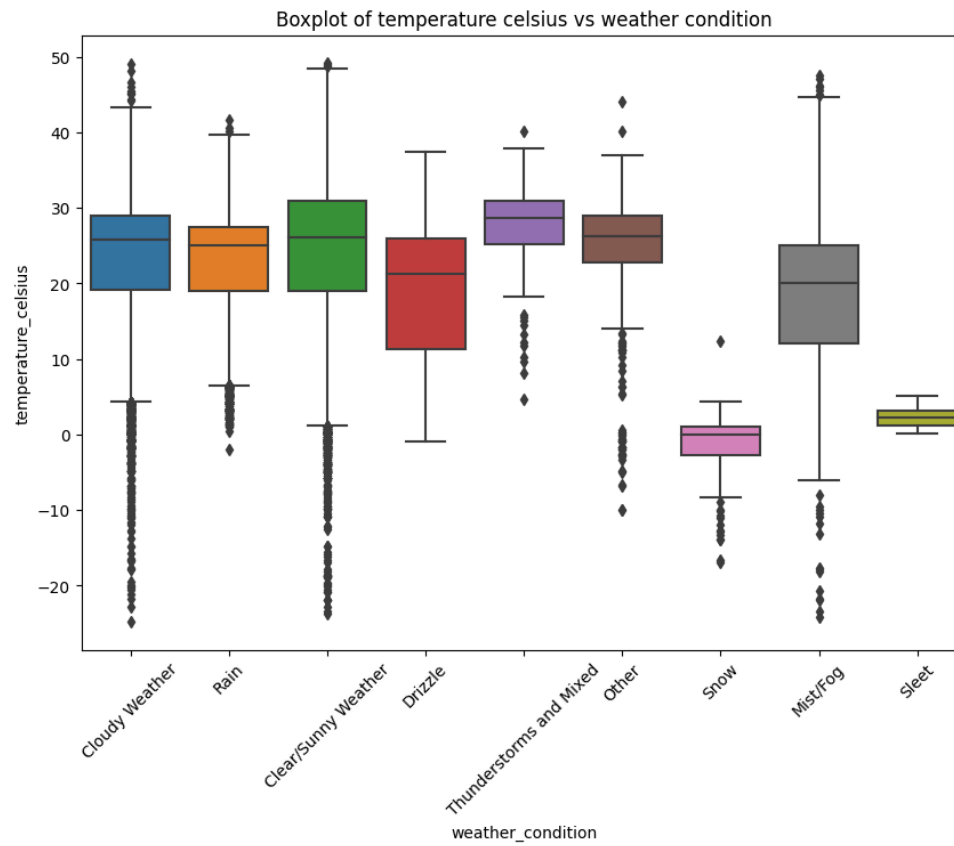
EDA

Correlation

A **correlation matrix** revealed that most variables in the dataset are not strongly correlated; however, some notable relationships emerged. **Temperature** showed a correlation with the difference between sunrise and sunset, as well as the UV index, while **humidity** was correlated with cloud coverage. Additionally, the **difference between sunrise and sunset** was correlated with the difference between moonrise and moonset, as well as the UV index. These relationships highlight key interactions within the dataset, providing valuable insights for feature selection and improving the effectiveness of predictive models.

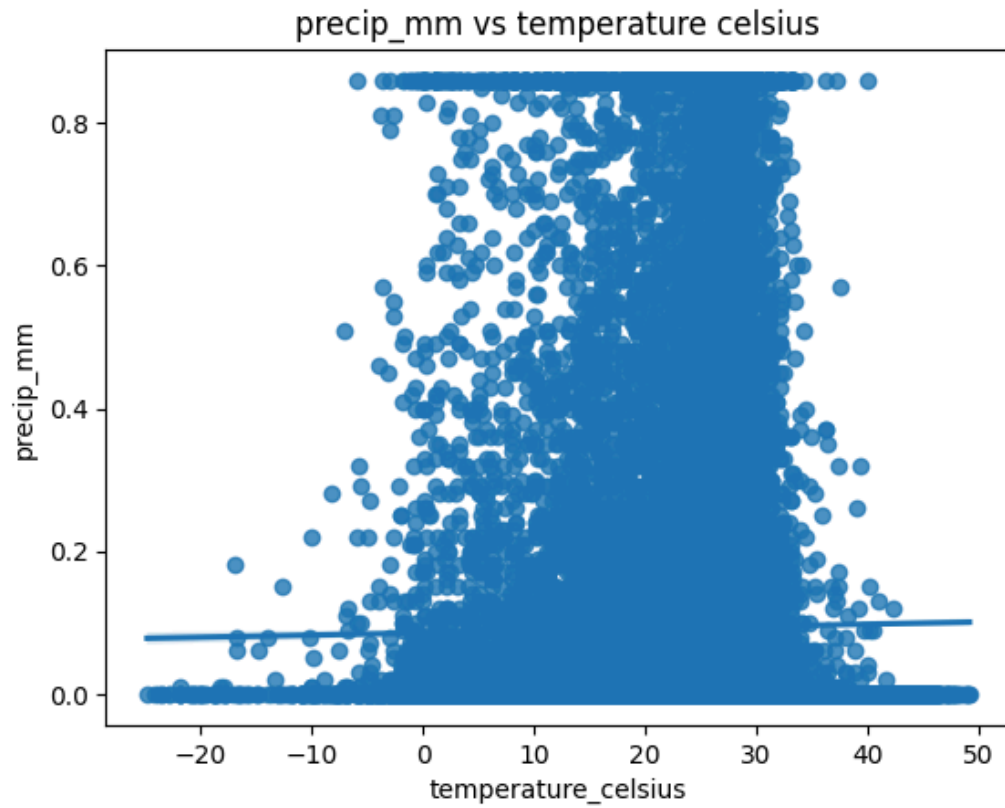


## Temperature vs weather condition



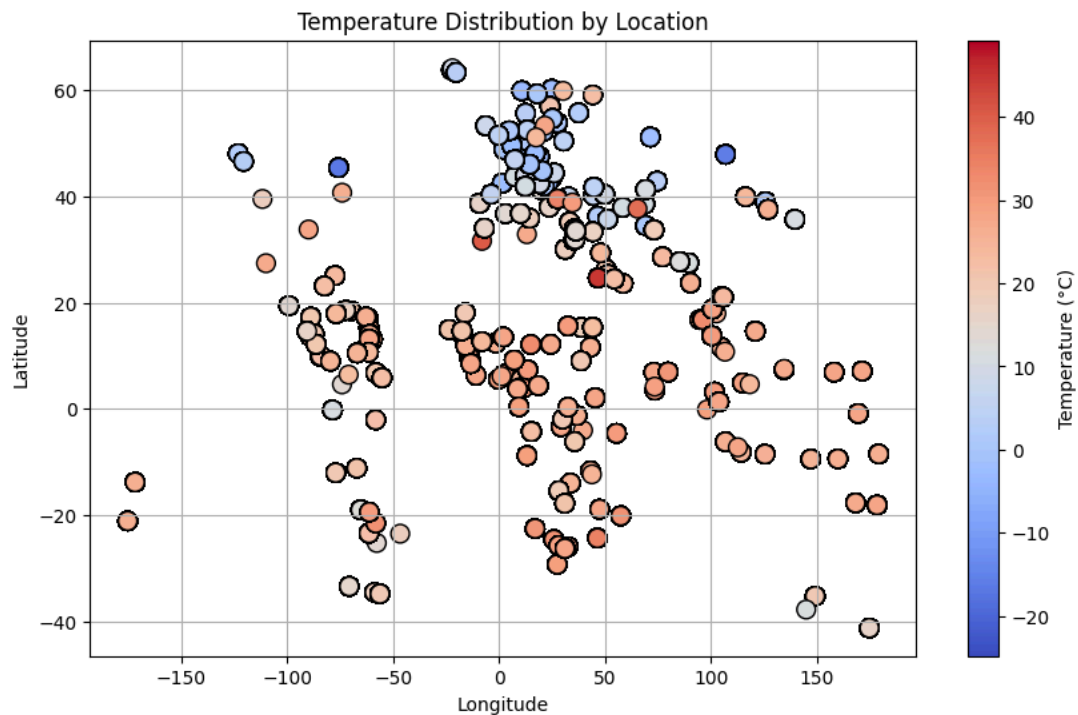
Through box plot temperature vs weather condition, I found the temperature of sunny and cloudy is irregular because they have a lot of negative outliers. This is because of different areas. Some areas are always in a cold temperature.

## Temperature VS Precipitation



I plotted the scatterplot of Precipitation vs temperature. We know the correlation between temperature and Precipitation is weak. This may indicate they don't have any connection that higher temperature leads to higher measurement in precipitation

## Spatial Analysis



I created a scatterplot of **latitude vs. longitude**, with data points color-coded in **blue** and **red** to represent weather conditions. **Red** indicates locations experiencing warm weather, while **blue** represents cold weather. From the plot, it is evident that locations with a latitude above 40° tend to experience colder weather. Additionally, I used the `groupby()` function to group the data by country and calculated the average temperature for each country. The analysis revealed that **55 countries are under cold weather**, while **155 countries are under warm weather**.

## Environmental Impact

I used `groupby()` to group data by weather condition and calculate the average air quality index per weather condition. The freezing drizzle condition has the worst air quality. The Blizzard has the best air quality.

## Model Building

### Modeling dataset

Since this is a forecasting project, I cannot use current information to predict the label, as the goal is to predict future conditions based on historical data. Each row in the dataset represents the information corresponding to the label, rather than leveraging past information.

To address this, I need to replace the current values with aggregated historical data. For instance, if Kabul recorded 22°C on June 11, 2024, I would calculate the average temperature over the previous 10 days and use that value instead. This approach ensures the model relies on prior information to forecast future weather conditions accurately.

To simplify the dataset and reduce its complexity, I grouped certain features into subcategories. For instance, I categorized **air quality index** values into **Low**, **Moderate**, **High**, and **Very High**, and **UV index** values into **No Protection Required**, **Protection Required**, and **Extra Protection Required**. Additionally, I grouped the numerous weather conditions into broader categories, such as **Rain**, **Snow**, **Sleet**, **Drizzle**, **Thunderstorm and Mixed**, **Clear/Sunny Weather**, **Cloudy Weather**, and **Mist/Fog**. To prepare the data for modeling, I applied **one-hot encoding** to transform these categorical features into binary numbers, ensuring compatibility with machine learning algorithms.

## Models

I split the dataset into 70% for training and 30% for testing. For this project, I employed **Random Forest**, **XGBClassifier**, and a **stacking model**. Initially, I ran **Random Forest** and **XGBClassifier** with **grid search** and **Principal Component Analysis (PCA)** to identify the optimal combination of hyperparameters and PCA components that maximized **accuracy**, **F1-score**, and **precision**. After determining the best configurations, I integrated these models into a **stacking method**, combining the strengths of the two ensemble models to generalize the results and enhance predictive performance.

### Random Forest

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.63      | 0.62   | 0.62     | 4905    |
| 1            | 0.57      | 0.14   | 0.23     | 1988    |
| 2            | 0.54      | 0.79   | 0.64     | 6271    |
| 3            | 0.62      | 0.13   | 0.22     | 961     |
| 4            | 0.00      | 0.00   | 0.00     | 131     |
| 5            | 0.00      | 0.00   | 0.00     | 132     |
| 6            | 0.00      | 0.00   | 0.00     | 9       |
| 7            | 0.00      | 0.00   | 0.00     | 209     |
| accuracy     |           |        | 0.57     | 14606   |
| macro avg    | 0.30      | 0.21   | 0.21     | 14606   |
| weighted avg | 0.56      | 0.57   | 0.53     | 14606   |

From the results of random forest, the accuracy was only 57% The true label accuracy from 0 to 3 was acceptable but from 5 to 7 did not look good because the Random Forest did not even capture trends of these types of weather conditions in the dataset. One possible reason I could think is that the training dataset does not include information from label 5 to 7.

## XGB

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.61      | 0.63   | 0.62     | 4905    |
| 1            | 0.45      | 0.25   | 0.32     | 1988    |
| 2            | 0.56      | 0.73   | 0.63     | 6271    |
| 3            | 0.54      | 0.13   | 0.20     | 961     |
| 4            | 0.00      | 0.00   | 0.00     | 131     |
| 5            | 0.00      | 0.00   | 0.00     | 132     |
| 6            | 0.00      | 0.00   | 0.00     | 9       |
| 7            | 0.00      | 0.00   | 0.00     | 209     |
| accuracy     |           |        | 0.57     | 14606   |
| macro avg    | 0.27      | 0.22   | 0.22     | 14606   |
| weighted avg | 0.54      | 0.57   | 0.54     | 14606   |

The XGB almost has the same result as Random Forest through Precision, recall, and accuracy metrics.

## Stacking Methods

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.65      | 0.61   | 0.63     | 4905    |
| 1            | 0.53      | 0.20   | 0.29     | 1988    |
| 2            | 0.55      | 0.78   | 0.64     | 6271    |
| 3            | 0.55      | 0.13   | 0.21     | 961     |
| 4            | 0.00      | 0.00   | 0.00     | 131     |
| 5            | 0.00      | 0.00   | 0.00     | 132     |
| 6            | 0.00      | 0.00   | 0.00     | 9       |
| 7            | 0.00      | 0.00   | 0.00     | 209     |
| accuracy     |           |        | 0.58     | 14606   |
| macro avg    | 0.28      | 0.22   | 0.22     | 14606   |
| weighted avg | 0.56      | 0.58   | 0.54     | 14606   |

The stack of Random Forest and XGB cannot improve anything.

**Final Thoughts**

Improving models require more time in feature engineering or simply getting more observation is a good method to try. Thank you for reading my report!